

# DataDeck

## Abstract Card Architecture

*Summary: Master Python's design patterns with abstract classes and interfaces by building a modular card system.*

*Version: 3.0*

# Contents


| | | |
|---|---|---|
| **I** | **Foreword** | **2** |
| **II** | **AI Instructions** | **3** |
| **III** | **Introduction** | **5** |
| **IV** | **General Instructions** | **6** |
| **V** | **Exercise 0: Creature Factory** | **7** |
| **VI** | **Exercise 1: Capabilities** | **9** |
| **VII** | **Exercise 2: Abstract Strategy** | **11** |
| **VIII** | **Submission** | **14** |

# Chapter I

# Foreword

*Gotta catch 'em all; but sometimes, the real treasure is the skills we made along the way.*

# Chapter II

# AI Instructions

## Context

During your learning journey, AI can assist with many different tasks. Take the time to explore the various capabilities of AI tools and how they can support your work. However, always approach them with caution and critically assess the results. Whether it's code, documentation, ideas, or technical explanations, you can never be completely sure that your question was well-formed or that the generated content is accurate. Your peers are a valuable resource to help you avoid mistakes and blind spots.

## Main message

- 👉 Use AI to reduce repetitive or tedious tasks.
- 👉 Develop prompting skills — both coding and non-coding — that will benefit your future career.
- 👉 Learn how AI systems work to better anticipate and avoid common risks, biases, and ethical issues.
- 👉 Continue building both technical and power skills by working with your peers.
- 👉 Only use AI-generated content that you fully understand and can take responsibility for.

## Learner rules:

- You should take the time to explore AI tools and understand how they work, so you can use them ethically and reduce potential biases.
- You should reflect on your problem before prompting — this helps you write clearer, more detailed, and more relevant prompts using accurate vocabulary.
- You should develop the habit of systematically checking, reviewing, questioning, and testing anything generated by AI.
- You should always seek peer review — don't rely solely on your own validation.

## Phase outcomes:

- Develop both general-purpose and domain-specific prompting skills.
- Boost your productivity with effective use of AI tools.
- Continue strengthening computational thinking, problem-solving, adaptability, and collaboration.

## Comments and examples:

- You'll regularly encounter situations — exams, evaluations, and more — where you must demonstrate real understanding. Be prepared, keep building both your technical and interpersonal skills.
- Explaining your reasoning and debating with peers often reveals gaps in your understanding. Make peer learning a priority.
- AI tools often lack your specific context and tend to provide generic responses. Your peers, who share your environment, can offer more relevant and accurate insights.
- Where AI tends to generate the most likely answer, your peers can provide alternative perspectives and valuable nuance. Rely on them as a quality checkpoint.

**✓ Good practice:**

> I ask AI: "How do I test a sorting function?" It gives me a few ideas. I try them out and review the results with a peer. We refine the approach together.

**✗ Bad practice:**

> I ask AI to write a whole function, copy-paste it into my project. During peer-evaluation, I can't explain what it does or why. I lose credibility — and I fail my project.

**✓ Good practice:**

> I use AI to help design a parser. Then I walk through the logic with a peer. We catch two bugs and rewrite it together — better, cleaner, and fully understood.

**✗ Bad practice:**

> I let Copilot generate my code for a key part of my project. It compiles, but I can't explain how it handles pipes. During the evaluation, I fail to justify and I fail my project.

# Chapter III

# Introduction

Welcome, aspiring Deck Architect, to the world of DataDeck!

Picture this: you're designing a creature-based card game inspired by popular monster-collecting games. Your cards are not just static objects; they are dynamic data entities that can be grouped into families, and that have capabilities used in a strategic way. But here's the challenge: how do you create a system flexible enough to handle thousands of different card types while maintaining clean, maintainable code?

The secret lies in **Abstract Programming Patterns**! You have already manipulated abstract classes and polymorphism in a previous project. We will now use more advanced patterns: abstract factories, extra capabilities and strategy patterns.

In this activity, you will manipulate cards in the Creature world to practice these advanced patterns. By the end, you'll think like a senior architect, designing systems that can evolve and improve over time.

# Chapter IV

# General Instructions

- Your project must be written in **Python 3.10 or later**.
- Your project must adhere to the **flake8** coding standard.
- All code must include comprehensive type annotations. Check this using `mypy`.
- All standard classes and collections are authorized, along with their methods (`int`, `str`, `list`, `dict`, etc.).
- All built-in functions are authorized, except for `eval()` and `exec()`.
- Your functions should handle exceptions gracefully to avoid crashes.
- External libraries are forbidden.



**PREREQUISITES**: This activity requires mastery of Python inheritance, abstract classes, polymorphism, and imports. We suggest that you complete the related projects before this one.

A `__init__.py` file is **MANDATORY** for each exercise folder. All testing code will be located at the root of the git repository.

# Chapter V

# Exercise 0: Creature Factory

|  | Exercise0 |
|---|---|
| ex0 | |
| Directory: *ex*0/ | |
| Files to Submit: `battle.py, ex0/ as a package with all needed files in it` | |
| Authorized: `builtins, standard types, import typing, import abc` | |

Let's start our game with the creation of basic Creature cards. But, as you may know, Creatures are classified into various families and can evolve. This exercise will focus on the *abstract factory* design pattern. You will implement the following elements in the `ex0/` folder:

- A `Creature` abstract class that holds attributes for the name and the type of the Creature, an abstract method `attack` and a concrete generic method `describe` that will return a standard message using the name and the type of the Creature.
- The following concrete classes that inherit from `Creature`: `Flameling`, `Pyrodon`, `Aquabub`, and `Torragon`. Their `attack` method will return an appropriate string message (see example).
- A `CreatureFactory` abstract class that will allow you to create the base Creature and the evolved Creature for the same family, using the `create_base` and `create_evolved` abstract methods.
- The concrete classes `FlameFactory` and `AquaFactory`, inheriting from `CreatureFactory`, that will handle the creation of the base and evolved Creature for each family (respectively `Flameling` and `Pyrodon` for `FlameFactory`, and `Aquabub` and `Torragon` for `AquaFactory`).
- Your `ex0` package cannot expose concrete Creature directly, it must only expose factories.

The `battle.py` script, at the root of the repository, will test the `ex0` package. The following scenario will be implemented:

- Instantiate the Flameling and Aquabub factories.
- Use a **single** function that receives a factory object and verifies that it can create the base and evolved Creature, and then each Creature can be described and can attack.
- Another function that receives both factories and makes base Creature fight.



You can use different Creatures, but you must keep the involved concepts (families and abstract factories).

Example:

```
$> python3 battle.py
Testing factory
Flameling is a Fire type Creature
Flameling uses Ember!
Pyrodon is a Fire/Flying type Creature
Pyrodon uses Flamethrower!

Testing factory
Aquabub is a Water type Creature
Aquabub uses Water Gun!
Torragon is a Water type Creature
Torragon uses Hydro Pump!

Testing battle
Flameling is a Fire type Creature
 vs.
Aquabub is a Water type Creature
 fight!
Flameling uses Ember!
Aquabub uses Water Gun!
```

# Chapter VI

# Exercise 1: Capabilities

|  | Exercise1 |
|---|---|
| ex1 | |
| Directory: *ex*1/ | |
| Files to Submit: `capacitor.py, ex1/ as a package with all needed files in it` | |
| Authorized: `builtins, standard types, import typing, import abc` | |

We will now add capabilities to our Creature! But maybe one day, these capabilities will not only apply to Creature. So we want to keep them separate: capability abstract classes will not inherit from the `Creature` base class!

In the `ex1/` folder/package, you will implement the following:

- A `HealCapability` abstract class that defines the `heal` abstract method. This method can take a "target" parameter if you wish.
- A `TransformCapability` abstract class that defines the `transform` and `revert` abstract methods. An attribute is used to make the state persistent and it impacts the `attack` implementation of Creature with this capability.
- Let's keep it simple: these methods will return simple strings that describe the action (just like the `attack` method).
- The following concrete classes that inherit from both `Creature` and `HealCapability`: `Sproutling` and `Bloomelle` that form a single family and will be available through the `HealingCreatureFactory` (inheriting from the `CreatureFactory`).
- The following concrete classes that inherit from both `Creature` and `TransformCapability`: `Shiftling` and `Morphagon` that form a single family and will be available through the `TransformCreatureFactory` (inheriting from the `CreatureFactory`).
- Again, your `ex1` package cannot expose concrete Creature directly, it must only expose factories.



You must, obviously, use the content of the `ex0` package and build this exercise upon it.

It is now time to define the test scenario. The `capacitor.py` script, located at the root of the Git repository, will proceed as follows:

- Create a healing Creature factory.
- Create the base, then the evolved Creature and make them: 1) be described; 2) attack; 3) heal.
- Create a transforming Creature factory.
- Create the base, then the evolved Creature and make them: 1) be described; 2) attack; 3) transform; 4) attack again; 5) revert.

Example:

```
$> python3 capacitor.py
Testing Creature with healing capability
 base:
Sproutling is a Grass type Creature
Sproutling uses Vine Whip!
Sproutling heals itself for a small amount
 evolved:
Bloomelle is a Grass/Fairy type Creature
Bloomelle uses Petal Dance!
Bloomelle heals itself and others for a large amount

Testing Creature with transform capability
 base:
Shiftling is a Normal type Creature
Shiftling attacks normally.
Shiftling shifts into a sharper form!
Shiftling performs a boosted strike!
Shiftling returns to normal.
 evolved:
Morphagon is a Normal/Dragon type Creature
Morphagon attacks normally.
Morphagon morphs into a dragonic battle form!
Morphagon unleashes a devastating morph strike!
Morphagon stabilizes its form.
```

# Chapter VII

# Exercise 2: Abstract Strategy

|  | Exercise2 |
|---|---|
| ex2 | |
| Directory: *ex2/* | |
| Files to Submit: `tournament.py, ex2/ as a package with all needed files in it` | |
| Authorized: `builtins, standard types, import typing, import abc` | |

As we have just experienced in the previous exercise, Creature with different capabilities will act differently. In our case, the healing family will `heal` after the `attack`, whereas when the transforming family will `transform`, then `attack`, then `revert`. The day we want to draw multiple Creature cards from various families and have them fight together in a tournament, we need battle code that is aware of each capability. Or maybe not? Let's explore the *abstract strategy* pattern! And yes, the day of the tournament has arrived.

In the `ex2/` folder/package, you will implement the following:

- A `BattleStrategy` abstract class that defines the `act` and `is_valid` abstract methods. The `is_valid` method returns a `bool` indicating that a `Creature` is suitable for the strategy, and the `act` method will be called by the tournament script.
- Three concrete classes that inherit from `BattleStrategy`:
  - The `NormalStrategy`, suitable for any Creature, that will simply use the `attack` method during the tournament.
  - The `AggressiveStrategy`, suitable for Creature with transform capabilities, that will `transform`, `attack`, and `revert` during the tournament.
  - The `DefensiveStrategy`, suitable for Creature with healing capabilities, that will `attack` and then `heal` during the tournament.
- In case an invalid strategy-Creature combination is tested, the `is_valid` method returns `False`. If the `act` method is called with an invalid combination, a dedicated exception is raised with a clear message.

We can finally create our tournament. The `tournament.py` script, located at the root of the Git repository, will proceed as follows:

- Create various Creature factories (from `ex0` and `ex1`).
- Create the three strategies.
- Define a **single** `battle` function that:
  - takes a list of opponents in the tournament; each opponent is defined as a tuple consisting of a `CreatureFactory` and a `BattleStrategy`.
  - makes each opponent fight once all other opponents.
  - organizes each fight using each Creature's associated strategy.
  - handles correctly invalid Creature-strategy tuples.

Example:

```
$> python3 tournament.py
Tournament 0 (basic)
 [ (Flameling+Normal), (Healing+Defensive) ]
*** Tournament ***
2 opponents involved

* Battle *
Flameling is a Fire type Creature
 vs.
Sproutling is a Grass type Creature
 now fight!
Flameling uses Ember!
Sproutling uses Vine Whip!
Sproutling heals itself for a small amount

Tournament 1 (error)
 [ (Flameling+Aggressive), (Healing+Defensive) ]
*** Tournament ***
2 opponents involved

* Battle *
Flameling is a Fire type Creature
 vs.
Sproutling is a Grass type Creature
 now fight!
Battle error, aborting tournament: Invalid Creature 'Flameling' for this aggressive strategy

Tournament 2 (multiple)
 [ (Aquabub+Normal), (Healing+Defensive), (Transform+Aggressive) ]
*** Tournament ***
3 opponents involved

* Battle *
Aquabub is a Water type Creature
 vs.
Sproutling is a Grass type Creature
 now fight!
Aquabub uses Water Gun!
Sproutling uses Vine Whip!
Sproutling heals itself for a small amount

* Battle *
Aquabub is a Water type Creature
 vs.
Shiftling is a Normal type Creature
 now fight!
Aquabub uses Water Gun!
Shiftling shifts into a sharper form!
Shiftling performs a boosted strike!
Shiftling returns to normal.

* Battle *
Sproutling is a Grass type Creature
 vs.
Shiftling is a Normal type Creature
 now fight!
Sproutling uses Vine Whip!
Sproutling heals itself for a small amount
Shiftling shifts into a sharper form!
Shiftling performs a boosted strike!
Shiftling returns to normal.
```

# Chapter VIII

# Submission

Turn in your assignment in your `Git` repository as usual. Only the work inside your repository will be evaluated during the defense. Do not hesitate to double-check the names of your files to ensure they are correct.



During evaluation, you may be asked to explain the design patterns explored in this project. Focus on understanding the concepts, not just the implementation.